ELEC 475: Lab 2 — Pre-Lab

Deep Learning Foundations: From LeNet to AlexNet

Arshia Esmail Tehrani and Michael Greenspan

27 September 2026

Table of Contents

Introduction 2
Task 1: Environment Isolation (~5 min) 2
Step 1: Discover Your Active Python (Before Creating `.venv`) 3
Step 2: Create the Dedicated `.venv` 4
Step 3: Inspect the Anchor Configuration (`pyvenv.cfg`) 4
Step 4: Activate the Virtual Environment 4
Step 5: Physically Explore Your Installed Packages in File Explorer 5
Task 2: Dataset Download (~10 min) 6
Task 3: Data Analysis (~10 min) 7
Task 4: Verification (~2 min) 8

Introduction

This Pre-Lab will fully prepare you for the Lab 2 in-lab session on Thursday. You will **not** write any code for this Pre-Lab. All scripts, dataset loaders, exploratory analysis tools, and verification routines are fully implemented and pre-packaged in the starter repository. The mandatory pre-lab deliverables consist solely of executing four terminal operation tasks prior to attending the in-lab session:

- Task 1: Environment Isolation
- Task 2: Dataset Download
- Task 3: Data Analysis
- Task 4: Verification

Task 1: Environment Isolation (~5 min)

Initialize `.venv` via the Conda base Python interpreter and install dependencies (`pip install -r requirements.txt`) (Section 2.1).

In Lab 1, you installed **Miniconda** or **Anaconda** as your base Python distribution. For Lab 2 and the remainder of ELEC 475, we enforce **strict environment isolation** using Python's built-in `venv` module anchored to your Conda Python installation.

Why Isolate with venv?

1. **Dependency Hygiene:** Deep learning packages (`torch`, `torchvision`, `torchinfo`, `matplotlib`) have complex sub-dependencies. An isolated `.venv` ensures that updates made for one lab never break code from previous labs.
2. **Reproducibility:** A project-local `.venv` makes your environment fully reproducible via `requirements.txt`.
3. **Portability:** Course TAs can immediately replicate your runtime environment during grading without guessing package versions.
4. **Physical Disk Transparency:** Unlike global installs where packages are scattered across root system folders, a `.venv` places every single downloaded package directly inside a local subfolder inside your project directory.
5. **HPC & Supercomputer Cluster Readiness:** In advanced research and high-performance computing (HPC) clusters—such as the **Digital Research Alliance of Canada** (formerly Compute Canada / Graham / Cedar / Narval)—heavy Conda environments create massive I/O bottlenecks on shared distributed filesystems (Lustre/NFS). Using lightweight, isolated Python `venv` environments is the strict standard for SLURM batch job submission. For students interested in scaling

machine learning workflows to supercomputers, see the setup guides in **`arshiatehrani/HPC-ML-Setup-Digital-Research-Alliance-Clusters`** and **`arshiatehrani/ssh-alliance`**.

Hands-On Task: Understanding Which Python You Are Using

A virtual environment (`.venv`) does not contain a full duplicate copy of Python's standard library or C-runtime. Instead, it **anchors** to whichever parent Python interpreter was active in your terminal when you ran `python -m venv .venv`.

Depending on your terminal state, there are **three common scenarios**:

Terminal State Prior to Creating `.venv`	Active Parent Interpreter	Where `.venv` Anchors Its Python Binary
Scenario A: Conda `base` Active *(Recommended)*	`(base)` prompt active	Points to Miniconda/Anaconda Python (e.g., `~/miniconda3/bin/python` or `C:\Users\username\miniconda3\python.exe`)
Scenario B: Custom Conda Environment Active	`(my_custom_env)` prompt active	Points to that custom Conda environment's Python binary
Scenario C: No Conda / Conda Deactivated	System default shell prompt	Points to the OS System Python (e.g., `/usr/bin/python3` or `C:\Program Files\Python311\python.exe`)

Step 1: Discover Your Active Python (Before Creating `.venv`)

Open your terminal and run the path discovery command for your platform:

```
# On macOS / Linux / WSL Bash (Course Standard):
which python
# Example Output (Conda base): /home/student/miniconda3/bin/python

# On Windows PowerShell:
where.exe python
# or: (Get-Command python).Source
# Example Output (Conda base): C:\Users\student\miniconda3\python.exe
```

Notice the exact path printed: this is the parent Python that will power your virtual environment.

Step 2: Create the Dedicated `.venv`

Navigate to your Lab 2 project root directory and initialize the environment:

```
# Navigate to lab folder
cd path/to/lab2_template

# Create the isolated environment
python -m venv .venv
```

Step 3: Inspect the Anchor Configuration (`pyvenv.cfg`)

Python creates a small configuration file at `.venv/pyvenv.cfg`. Open and inspect it:

```
# On macOS / Linux / WSL Bash:
cat .venv/pyvenv.cfg

# On Windows PowerShell:
Get-Content .venv\pyvenv.cfg
```

You will see:

```
home = C:\Users\student\miniconda3
include-system-site-packages = false
version = 3.11.9
```

Notice the `home` field: it explicitly records the parent Python directory identified in Step 1. Setting `include-system-site-packages = false` guarantees that packages installed in your base Conda environment will **not** leak into your lab environment.

Step 4: Activate the Virtual Environment

Activate the environment to redirect your shell's path:

```
# On macOS / Linux / WSL Bash (Course Standard):
source .venv/bin/activate

# On Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# On Windows Command Prompt (CMD):
.\.venv\Scripts\activate.bat
```

[!TIP] **Verification:** Your terminal prompt will now be prefixed with `(.venv)`.
Re-run `which python` (WSL/Bash) or `where.exe python` (PowerShell).
The very first path listed is now your project-local `.venv` executable (e.g., `.../lab2_template/.venv/bin/python` or

`...\lab2_template\.venv\Scripts\python.exe`), proving that all subsequent Python and pip executions are safely quarantined.

Hands-On Task: Visualizing What `pip install` Actually Does

When you execute `pip install -r requirements.txt`, pip downloads pre-compiled wheel files (`.whl`), unzips them, and writes pure Python modules and compiled C/CUDA extensions (`.pyd` on Windows, `.so` on Linux) directly into your local disk directory.

```
# Upgrade package installer
python -m pip install --upgrade pip

# Install pinned Lab 2 dependencies
pip install -r requirements.txt
```

Step 5: Physically Explore Your Installed Packages in File Explorer

To demystify package management, open the physical folder on disk where your deep learning libraries are installed:

- **Windows Users (PowerShell):** Run this command to open the folder directly in Windows File Explorer:

  ```
  explorer .venv\Lib\site-packages
  ```

- **WSL Users (Windows Subsystem for Linux):** WSL allows launching Windows File Explorer directly from your Linux Bash terminal:

  ```
  explorer.exe .venv/lib/python3.*/site-packages
  # Or view in terminal:
  ls -la .venv/lib/python3.*/site-packages/
  ```

- **macOS / Linux Users:**

  ```
  # macOS:
  open .venv/lib/python3.*/site-packages
  # Linux:
  xdg-open .venv/lib/python3.*/site-packages || ls -la
  .venv/lib/python3.*/site-packages/
  ```

In File Explorer or your terminal listing, you can physically inspect the installed packages: - `torch/`: The PyTorch core engine containing `__init__.py`, `nn/`, and the C++ CUDA backend dynamic link libraries (`torch_cpu.dll` or `libtorch_cpu.so`). - `torchvision/`: The computer vision extension containing datasets, transforms, and image I/O routines. - `numpy/`: High-performance multi-dimensional array operations. - `matplotlib/`: The visualization engine. - `torchinfo/`: The neural network model summary utility. - `*.dist-info/` folders: Metadata manifests documenting exact versions, licenses, and installed file lists.

Task 2: Dataset Download (~10 min)

Download and extract **both** CIFAR-10 (`./data/cifar-10-batches-py/`) and Tiny ImageNet-200 (`./data/tiny-imagenet-200/`) Both datasets are required for the in-lab and post-lab curricula.

In Lab 2, our data loader automatically fetches datasets via PyTorch `torchvision`. Both datasets are also available through their official creators and the **Hugging Face Hub**:

1. **CIFAR-10 (Primary In-Lab & Ablation Dataset):**
   - **Creators:** Alex Krizhevsky, Vinod Nair, & Geoffrey Hinton (University of Toronto, 2009).
   - **Official Website:** cs.toronto.edu/~kriz/cifar.html
   - **Hugging Face Hub:** `huggingface.co/datasets/cifar10`
   - **Description:** 60,000 32 × 32 color images across 10 classes (50,000 train, 10,000 test). Upscaled to 64 × 64 × 3 RGB.
   - **Why Selected for Lab 2:** Instant download via `torchvision.datasets.CIFAR10(download=True)`. Five training epochs execute in under 3 minutes on GPU, guaranteeing completion within the 2-hour lab while showing a clear accuracy jump from LeNet (≈ 60%) to AlexNet (≈ 80%).

2. **Tiny ImageNet-200 (Secondary Benchmark Dataset):**
   - **Origin:** Stanford University CS231n Computer Vision Challenge.
   - **Official Archive:** cs231n.stanford.edu/tiny-imagenet-200.zip
   - **Hugging Face Hub:** `huggingface.co/datasets/zh-plus/tiny-imagenet`
   - **Description:** 100,000 training images, 10,000 validation images, native 64 × 64 × 3 RGB across 200 WordNet synset classes.
   - **Why Selected for Lab 2:** Directly provides native 64x64 ImageNet resolution to test how deeper networks scale to high class diversity.

Task 3: Data Analysis (~10 min)

Run `python utils/eda.py --dataset both` to compute dataset statistics, verify class balance, analyze channel pixel dynamics, and generate diagnostic artifacts in `./plots/` for both datasets.

You do **not** write any code for this task. The complete EDA suite is 100% written, tested, and pre-packaged for you in `utils/eda.py`. Your task is simply to run the script, examine the terminal statistics, and view the generated figures in `./plots/` to familiarize yourself with the CIFAR-10 and Tiny ImageNet-200 datasets before class.

To run EDA on CIFAR-10:

```
python utils/eda.py --dataset cifar10
```

To run EDA on both CIFAR-10 and Tiny ImageNet-200 simultaneously with a comparative summary:

```
python utils/eda.py --dataset both
```

What `utils/eda.py` Profiles & Computes Across Both Datasets:

1. **Dataset Inventory & Storage Footprint:**
   - Total sample counts (50,000 train for CIFAR-10, 100,000 train for Tiny ImageNet-200).
   - Class count: $C = 10$ vs. $C = 200$.
   - Native resolution (32×32 vs 64×64) and standardized tensor shape ($3 \times 64 \times 64$, float32 in [0.0,1.0]).
   - On-disk storage footprint (approx. 170 MB vs 240 MB).

2. **Pillar 1: Class Balance, Frequency Profiling, Shannon Entropy & Imbalance Diagnostics:**
   - Per-class counts and percentage shares.
   - **Imbalance Ratio ($IR = \frac{\max N_c}{\min N_c}$):** Quantifies whether categories are equally represented. For CIFAR-10 ($IR = 1.00$) and Tiny ImageNet ($IR = 1.00$), the datasets are perfectly balanced.
   - **Shannon Entropy of Class Distribution:**

$$H = -\sum_{c=1}^{C} p_c \log_2(p_c)$$

     Compares empirical entropy H against theoretical maximum $H_{\max} = \log_2(C)$ to calculate the Entropy Ratio $\frac{H}{H_{\max}}$. For uniform distributions, $\frac{H}{H_{\max}} = 1.0000$.

- **Gini Impurity Index:** $G = 1 - \sum_{c=1}^{C} p_c^2$.
- **Automated Metric Recommendation:** Confirms whether Top-1 Accuracy is safe or if Macro F_1, Balanced Accuracy, or Recall must be prioritized to guard against the Accuracy Paradox.
- Generates and saves the class distribution plot to `plots/eda_<dataset>_class_distribution.png`.

3. **Pillar 2: Channel-Wise Pixel Intensity Dynamics (RGB Statistics & Histograms):**
   - Empirical dataset-wide channel means (μ_R, μ_G, μ_B) and standard deviations $(\sigma_R, \sigma_G, \sigma_B)$.
   - Global dynamic range (confirms strictly [0.0000,1.0000]).
   - Channel intensity percentiles $(P_1, P_{25}, P_{50}, P_{75}, P_{99})$.
   - **Inter-Channel Pearson Correlation Matrix:** Measures chromatic redundancy (e.g. correlation between Red and Green channels $\approx$ 0.85–0.90).
   - Generates 50-bin density histograms across Red, Green, and Blue channels and saves to `plots/eda_<dataset>_channel_histograms.png`.
   - Outputs the exact `transforms.Normalize(...)` block to paste into code.

4. **Pillar 3: Data Quality, Integrity & Outlier Diagnostics:**
   - Corrupt file decode check: verifies zero decode errors.
   - NaN / Inf representation check: verifies zero corrupted floating-point values.
   - **Dead / Blank Image Detection:** Computes per-image spatial variance Var(I). Flags any blank or uninformative frames ($\sigma^2 < 10^{-4}$).
   - Extreme brightness outlier counts: counts images with mean brightness < 0.05 (extreme underexposure) or > 0.95 (extreme saturation).

5. **Pillar 4: Stratified Visual Inspection Sample Grid:**
   - Extracts a diverse, multi-class grid of images across categories.
   - Renders image samples with human-readable ground-truth class labels.
   - Saves high-resolution sample grid to `plots/eda_<dataset>_sample_grid.png`.

Task 4: Verification (~2 min)

Execute `python prelab_verify.py` (Section 11.1) to validate Python environment configuration, CUDA GPU acceleration, 4D tensor shapes `(B, 3, 64, 64)`, dataset directories, and generated EDA artifacts.

When `prelab_verify.py` completes with all checks passing, your environment is certified ready for the in-lab session.